Введение в MATLAB

1. Знакомство с MATLAB

1.1. Что такое MATLAB?

MATLAB (сокращение от англ. «*Matrix Laboratory*», в русском языке произносится как Матла́б) – пакет прикладных программ для решения задач технических вычислений и одноимённый язык программирования, используемый в этом пакете. MATLAB используют более 1 000 000 инженерных и научных работников, он работает на большинстве современных операционных систем, включая Linux, Mac OS, Solaris (начиная с версии R2010b поддержка Solaris прекращена) и Microsoft Windows.

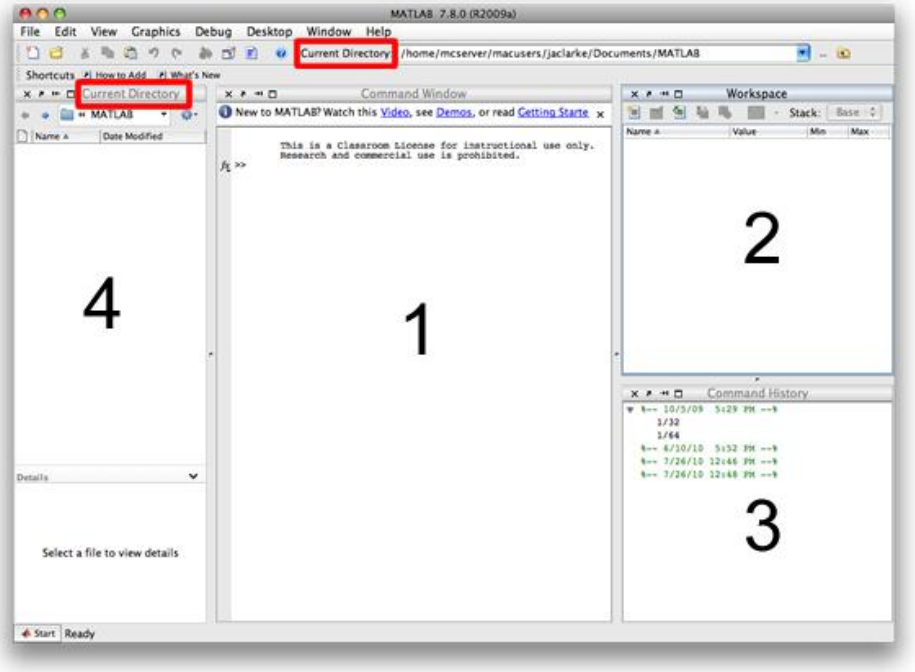
MATLAB:

- ✓ является высокопроизводительным, высокоуровневым языком;
- ✓ означает Матричную Лабораторию;
- ✓ может быть использован в качестве причудливого калькулятора;
- ✓ позволяет с легкостью работать с множеством матрицами, а не с одним числом одновременно;
- ✓ полезен для всего, что требует матричных и векторных операций, таких как:
  - математические, научные, инженерные, статистические и финансовые проблемы;
  - все, что требует построения/визуализации и анализа данных;
- ✓ поставляется с базовым набором инструментов для визуализации данных и для выполнения вычислений с матрицами и векторами
  - Для конкретных технологий, MATLAB предоставляет набор инструментов, которые добавляются к базовой функциональности MATLAB. В основном будет использован модуль Image Processing Toolbox. Некоторые другие инструментарии включают в себя:
    - Statistics Toolbox;
    - Neural Network Toolbox;
    - Fuzzy Logic Toolbox;
    - Signal Processing Toolbox;
    - Wavelet Toolbox;
    - Financial Toolbox;
    - Bioinformatics Toolbox;
    - Database Toolbox.

1.2. Графический пользовательский интерфейс MATLAB

Графический интерфейс MATLAB написана на языке Java и должен быть похож на любые ОС. Он разделен на 4 основные части:

1. Командное окно ***Command Window*** – окно, в котором вводятся команды. Вывод или сообщения об ошибках, как правило, появляются и здесь.
2. Окно рабочей области ***Workspace*** – когда определяются новые переменные, они будут перечислены в этом окне.
3. Окно предыстории ***Command History*** – окно, в котором запоминаются введенные ранее команды. Если необходимо повторно запустить предыдущую команду или ее изменить, то можно ее перетащить из этого окна на окно командной строки или снова запустить ее двойным щелчком.
4. Информационное окно ***Current Directory*** – показывает файлы в Текущей Директории.



2. Основные операции

2.1. Создание Матриц

Все в системе MATLAB сохраняется в виде матриц или массивов. Для создания массива размерностью 1×1 или скаляра следует использовать:

```
a=5
```

Для проверки того, что a представляет собой массив размерностью 1×1, необходимо выполнить команду:

```
whos
```

которая отображает список всех переменных с указанием размера и объема занимаемой ими памяти.

Для создания матрицы 4×4, используется следующий синтаксис:

```
b = [1 2 3 4; 5 6 7 8; 9 10 11 12; 13 14 15 16]
```

или

```
b = [
      1  2  3  4
      5  6  7  8
      9 10 11 12
     13 14 15 16
]
```

- ✓ открытие и закрытие квадратных скобок необходимо для матриц размерностью больше 1×1;
- ✓ записи в строке разделены пробелами или запятыми;
- ✓ точка с запятой (;) означает знак конца строк в матрице;
- ✓ если любая операция заканчивается знаком точки с запятой (;), то это указывает системе MATLAB на подавление вывода на экран результата выполнения введенной команды;

- ✓ если нужно вспомнить, что содержится в какой-либо переменной, например в `b`, следует набрать `b` ↵ ;

Можно также использовать MATLAB функции для создания матриц. В следующей таблице приведены некоторые из этих функций:

Функция	Описание
`zeros(i,j)`	создает матрицу нулей размерностью `i×j`
`ones(i,j)`	создает матрицу единиц размерностью `i×j`
`rand(i,j)`	создает матрицу случайных элементов (между 0 и 1) размерностью `i×j`
`randn(i,j)`	создает матрицу случайных элементов размерностью `i×j`, подчиняющихся нормальному распределению со средним значением 0 и стандартным отклонением 1
`eye(i)`	создает единичную матрицу размерностью `i×i` (матрица нулей с единицами по диагонали)

В приведенной выше таблице, `i` – это количество строк и `j` – число столбцов. Будьте осторожны, это может сбить вас с толку, если вы привыкли думать в терминах *x* и *y*.

Если имеется большое количество данных, можно хранить данные в файле и загрузить его в MATLAB. Когда набирается содержимое файла, каждое значение столбца разделяется горизонтальным пробелом и каждая строка начинается с новой строки. Потребуется одинаковое количество элементов в каждой строке. Например, можно создать текстовый файл (так называемый `mydata.dat`) со следующим содержимым:

```
5.0    3.2   6.8
1.4   12.6   3.9
7.1   16.4   8.3
9.3    5.2   4.7
```

Чтобы прочитать этот файл и сохранить его в переменной `MyData`, можно использовать следующую команду:

```
load myData.dat
```

2.2. Основные операции с Матрицами

Учитывая `b` (см. ниже), в следующей таблице приведены некоторые основные операции с матрицами и результаты этих функций:

```
b =
    1    2    3    4
    5    6    7    8
    9   10   11   12
   13   14   15   16
```

Функция	Описание	Результат
`sum(b)`	Суммирует значения по каждому столбцу*	`ans =` `28  32  36  40`

`diag(b)`	Возвращает все элементы по диагонали	`ans =`
		`1`
		`6`
		`11`
		`16`

`b'`	Оператор транспонирования. Первый столбец становится первой строкой, второй столбец становится второй строкой и так далее	`ans =`
		`1  5  9  13`
		`2  6  10  14`
		`3  7  11  15`

		4 8 12 16
`max(b)`	Возвращает максимальные значения по каждому столбцу*	`ans =` `13 14 15 16`
`min(b)`	Возвращает минимальные значения по каждому столбцу*	`ans =` `1 2 3 4`
`mean(b)`	Возвращает среднее по каждому столбцу	`ans =` `7 8 9 10`
`mean2(b)`	Возвращает среднее всех элементов в матрице	`ans =` `8.5000`

*Хорошо иметь представление о максимуме, минимуме или сумме по каждому столбцу, но иногда они необходимы для всей матрицы. Если имеется многомерная матрица, можно преобразовать ее в матрицу с одним столбцом, используя нотацию `b(:)` (обсуждается ниже в секции «Оператор двоеточие»). Например, чтобы получить сумму всех значений в `b`, надо написать:

```
sum(b(:))
```

которая генерирует следующий результат:

```
ans =
      136
```

2.3. Индексы

Учитывая `b` (см. ниже), если требуется обратиться к элементу в первой строке и четвертом столбце, можно использовать следующую нотацию:

```
b(1,4)
```

Результат следующий:

```
ans =
      4
```

Обратите внимание на две вещи:

- ✓ не начинайте с индекса 0;
- ✓ и в следующем порядке `row, column` формат.

Можно использовать один индекс. В этом случае MATLAB представляет матрицу как один длинный столбец, который является объединением каждого исходного столбца. Так, первые четыре вхождения в `b`, когда используется один индекс, являются: `1 5 9 13`. Таким образом, чтобы обратиться к тому же элементу, что и выше, `b(1,4)`, используя единичный индекс, можно написать: `b(13)`.

MATLAB обеспечивает проверку ошибок в случаях, когда идет обращение к матрице с индексами, превышающих предельные. Например, набрав: `b(1,5)` приводит к следующему:

```
??? Index exceeds matrix dimensions.
```

Однако можно присвоить значение позиции, что превышает размерность матрицы. Например, если использовать `b(1,5) = 17`, получим следующую матрицу:

```
b =
     1     2     3     4    17
     5     6     7     8     0
     9    10    11    12     0
    13    14    15    16     0
```

В этом случае, новый (пятый) столбец будет создан и он будет заполнен нулями где не были указаны значения.

2.4. Оператор двоеточие

Оператор двоеточие (`:`) может быть использован для выбора блоков или диапазонов значений.

Введя `b` и нажав клавишу ввода можно увидеть значения в `b`:

```
b =
     1     2     3     4
     5     6     7     8
     9    10    11    12
    13    14    15    16
```

Если необходимо только подмножество `b`, например, внутренний квадрат, можно использовать следующую команду: `b(2:3,2:3)`

```
ans =
     6     7
    10    11
```

Кроме этого, оператор двоеточия создает вектор-строку, содержащий числа в указанном диапазоне. Например, введя `2:8` приведет к:

```
ans =
     2     3     4     5     6     7     8
```

Можно также определить инкремент или декремент (как средний аргумент) с оператором двоеточия. Например, введя `1:2:8` приведет к :

```
ans =
     1     3     5     7
```

В этом случае инкрементом является 2. По сути, возвращаются все нечетные номера от 1 до 8.

Просто чтобы дать больше примеров, в следующей таблице приведены результаты использования оператора двоеточия, использующегося как в качестве индекса к `b`, так и самого по себе:

Выражение	Результат	Описание
`5:10`	`ans =` `5 6 7 8 9 10`	Значения от 5 до 10.
`10:-1:5`	`ans =` `10 9 8 7 6 5`	Значения от 10 до 5 (используя декремент 1).

`b(1:2,3:4)`	`ans =`
	`3 4`
	`7 8`
		Строки 1-2, столбцы 3-4 (возвращает значения в квадрате, находящегося в правом верхнем углу `b`).

`b(5:8)`	`ans =` `2 6 10 14`	Это использование нотации с одним индексом. Помните, что это объединение всех столбцов в один (один за другим). В этом случае получаем значения второго столбца.
`b(end)`	`ans =` `16`	`end` **является специальным значением, которое указывает на последний элемент матрицы**. Это нотация с одним индексом. Можно также записывать `b(end,end)` для обозначения конечной строки и конечного столбца.
`b(1:4,4)`	`ans =`	Это получение значений последнего

	4
	8
	12
	16
		столбца (строки с 1 по 4 и столбец 4). Когда необходимо определить все строки и последний столбец, можно использовать следующее обозначение:
		`b(1:end,end)`
		или оператор двоеточия самого по себе, который определяет весь диапазон строк
		`b(:,end)`

`b(end:-1:1,:)`	`ans =` 13 14 15 16 9 10 11 12 5 6 7 8 1 2 3 4	Изменяет порядок строк так, что последняя строка станет первой и первая строка станет последней. Понимайте это так: строки с последней до первой с декрементом 1 (`end: -1:1`) и все столбцы (`:`).
`b(:)`	`ans =` 1 5 9 13 2 6 10 14 3 7 11 15 4 8 12 16	Производит один длинный столбец, который представляет собой объединение отдельных столбцов. Это полезно при использовании функций, таких как максимум, минимум, или сумма.

2.5. Переменные

- ✓ Когда объявляется переменная, не нужно указывать тип или размерность (вспомните то, как это можно сравнить с созданием массива в C++).
- ✓ Если существует еще один оператор присваивания, использующий то же самое имя переменной, то система MATLAB изменит ее содержимое и, при необходимости, выделит новую память.
- ✓ Имена переменных чувствительны к регистру (`b` и `B` две разные переменные).
- ✓ Имена переменных состоят из символов, состоящие из любого количества символов, цифр или символов подчеркивания. Обратите внимание, что нельзя использовать специальные символы в именах переменных, например, знак доллара ($), как в некоторых других языках.
- ✓ Имена переменных могут быть любой длины, но только первые `N` символов в имени будут использоваться. Для того чтобы узнать чему равно `N` надо ввести:

```
namelengthmax
```

Каждая объявленная переменная является массивом/матрицей. Например, `myVar = 2` создает матрицу размерностью 1×1 с единственным элементом (`2`).

2.6. Предопределенные Функции и Помощь

Базовый пакет MATLAB предоставляет ряд функций, которые полезны для работы с числами и массивами. Все следующие функции являются частью основы MATLAB: `sin`, `cos`, `sqrt`, `abs`, `ceil`, `isprime`, `gsd`, `lcm`, `find`, `rot90`, `length`, `isempty` и `isequal`.

Для более обширного перечня некоторых функций, доступных в базовом пакете MATLAB, можно ввести следующие команды:

`help elfun`	(подробнее об элементарных математических функциях)
`help specfun`	(подробнее о специализированных математических функциях)
`help elmat`	(подробнее об элементарных матрицах и операциях с матрицами)

Список функций, доступных в Image Processing Toolbox системы MATLAB, можете вывести следующей командой:

`help images`	(подробнее об Image Processing Toolbox)

Если необходимо больше помощи по конкретной функции, можно набрать:

```
help functionName
```

Например, если нужно узнать больше о sin функции, можно набрать:

```
help sin
```

или еще лучше:

```
doc sin
```

что открывает индексированное окно помощи с более подробной информацией о `sin`.

`sin` является встроенной функцией, которая означает, что она является частью ядра MATLAB и не возможно увидеть код этой функции. В отличие от этого, существуют такие функции, как `acosd`, которые реализованы в M-файлах. Файл (`acosd.m`) хранится в: `matlabroot\toolbox\matlab\elfun` (где `matlabroot` это каталог установки MATLAB). Можно просмотреть этот файл непосредственно из MATLAB, используя:

```
type acosd.m  (typing .m, in this case, is optional)
```

или отредактировать его, если ввести:

```
edit acosd
```

2.7. M-файлы

Команда редактирования в предыдущем разделе показывает очень простой M-файл для `acosd` функции. Следующее предназначено для обобщения M-файлов:

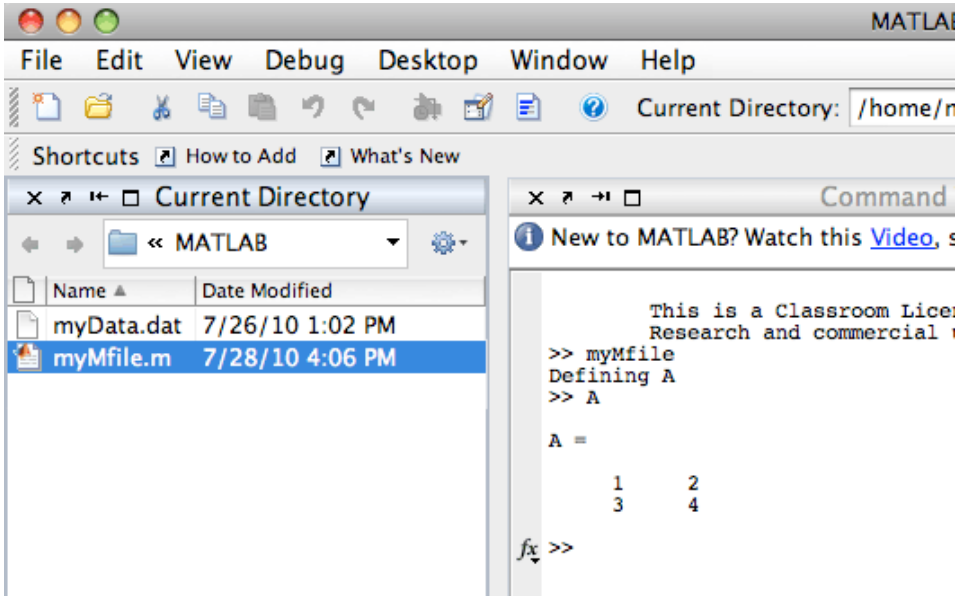
- ✓ M-файлы являются текстовыми файлами с MATLAB кодом;
- ✓ M-файлы имеют расширение `*.m`;
- ✓ Упражнение: Создать простой M-файл, который работает как скрипт для создания массива с именем `A`.
  - Используется простой текстовый редактор, например Блокнот для создания следующего текстового файла:

```
%{
   My first M-file
   Author: me
   Date: today
%}

% Create a Matrix
disp('Defining A');
```

```
A = [
    1 2
    3 4
  ];
```

- Обратите внимание на комментарии:
  - Однострочные комментарии начинаются с `%`;
  - Многострочные комментарии заключаются в блок `%{ %}`.
- Обратите внимание на точку с запятой в конце двух команд. Обычно точка с запятой предотвращает вывод результата выполнения команды, но `disp` работает так или иначе.
- Сохраняется файл как `myMfile.m` в текущей директории MATLAB
- Можно увидеть `myMfile.m` как показано на рисунке ниже.
  - **Совет:** Следует убедиться в том, что все файлы ресурсов сохранены в текущем каталоге, в противном случае, MATLAB не найдет их при запуске.



- Файл запускается следующим образом:

```
myMfile
```

- Теперь массив `A` должен быть создан.

2.8. Конкатенация

Оператор конкатенации обозначается квадратными скобками `[]`. Даже не зная об этом, можно массивы размерности 1×1 объединить в многомерные массивы с оператором, подобным следующему, который создает массив размерности 2×2 с именем `A`:

```
A=[1  2; 3  4]
```

Для создания массива, размерности 4×4, который повторяет элементы `A` три раза, можно использовать:

```
[A A; A A]
```

что приводит к:

```
ans =
     1     2     1     2
     3     4     3     4
     1     2     1     2
```

```
     3     4     3     4
```

Можно иметь два экземпляра `A` (один под другим) и в то же время перевернуть второй экземпляр так:

```
[A; A(end:-1:1,:)]
```

что приводит к:

```
ans =
     1     2
     3     4
     3     4
     1     2
```

2.9. Изменение Матриц

Допустим, что `b` было создано, как показано ниже:

```
b =
     1     2     3     4
     5     6     7     8
     9    10    11    12
    13    14    15    16
```

Если необходимо изменить элемент, можно сделать это с помощью индексов и назначения как:

```
b(3,1)=3
```

что приводит к:

```
b =
     1     2     3     4
     5     6     7     8
     3    10    11    12
    13    14    15    16
```

Можно изменить всю строку с указанием:

```
b(3,:)=3
```

что приводит к:

```
b =
     1     2     3     4
     5     6     7     8
     3     3     3     3
    13    14    15    16
```

Можно заменить значения в строке или столбце, используя один одномерный массив с точно правильной размерностью. Например, чтобы заменить вторую колонку `b` можно написать:

```
b(:, 2) = [1 2 3 4]
```

что приводит к:

```
b =
     1     1     3     4
     5     2     7     8
     3     3     3     3
    13     4    15    16
```

Также можно заменить подмножество значений в строке или столбце следующим образом:

```
b(3,1:3) = [1 2 3]
```

что приводит к:

```
b =
     1     1     3     4
     5     2     7     1
     1     2     3     3
    13     4    15    16
```

Если нужно удалить что-нибудь, можно использовать `[]` (пустые квадратные скобки). Например, чтобы удалить третий ряд `b`, можно написать:

```
b(3,:)=[]
```

что приводит к:

```
b =
     1     1     3     4
     5     2     7     8
    13     4    15    16
```

Тем не менее, нельзя частично удалить или заменить строки или столбец. Например, следующие операторы:

```
b(1:2,2)=[]
b(3,2) = []
```

приведут к следующей ошибке:

```
??? Subscripted assignment dimension mismatch.
```

Можно, однако, удалить каждый четвертый элемент и преобразовать оставшиеся элементы в строку матрицы, как в:

```
b(1:4:end)=[]
```

что приводит к:

```
b =
     5     3     2     3     3     7     4     8     3
```

2.10. Операторы

В MATLAB существует и матричные операции, и операции над массивами. Сложение (`+`) и вычитание (`-`) как для массивов, так и для матриц выполняются элемент за элементом. С другими операциями, например умножения, линейная алгебра определяет как будут умножаться матрицы; это не поэлементное умножение.

Иногда, можно использовать поэлементные операции над массивами. MATLAB отделяет матричные операций от операций над массивами с использованием точечной нотации перед операторами. Ниже приведен список и описание операций над массивами. Соответствующие операции над матрицами точку не используют.

`A*B`	**Умножение матриц.** Если одна из сторон является скалярной все элементы другой стороны умножаются на скаляр. В противном случае выполняется умножение матриц, и количество столбцов в `A` должно совпадать с количеством строк в `B`.

Смотреть: `help mtimes`

Умножение массивов.

`A.*B`

Если одна из сторон является скалярной все элементы другой стороны умножаются на скаляр.
В противном случае соответствующие элементы каждой стороны перемножаются между собой, и обе стороны должны быть матрицами одинаковой размерности.

Смотреть: `help times`

Правое деление матриц.

`A/B`

Если `B` является скаляром все элементы `A` делятся на этот скаляр.
В противном случае, если возможно, выполняется следующее эквивалентное умножение: AB^{-1}
В частности эта операция выполняется: `(B'\A')'`

Смотреть: `help mrdivide`

Правое деление массивов.

`A./B`

Если `B` является скаляром все элементы `A` делятся на этот скаляр.
В противном случае, элементы `A` делятся на соответствующие элементы в `B` и обе стороны должны быть матрицами одинаковой размерности

Смотреть: `help rdivide`

Левое деление матриц.

`A\B`

Если `A` является скаляром все элементы `B` делятся на этот скаляр.
В противном случае, если возможно, выполняется следующее эквивалентное умножение: $A^{-1}B$. Будет выведено предупреждение если результат будет вероятно недостоверным.

Смотреть: `help mldivide`

Левое деление массивов.

`A.\B`

Если `A` является скаляром все элементы `B` делятся на этот скаляр.
В противном случае, элементы `B` делятся на соответствующие элементы в `A` и обе стороны должны быть матрицами одинаковой размерности

Смотреть: `help ldivide`

Возведение в степень матрицы.

`A^B`

Если `A` является матрицей, она должна быть квадратной и `B` должно быть скаляром. Тогда `A^3` будет эквивалентно `AAA`.
Если `B` является матрицей, она должна быть квадратной и `A` должно быть скаляром. Результат вычисляется как функция от `B` с использованием собственных чисел и собственных векторов. Результат операции – квадратная матрица.

Смотреть: `help mpower`

Возведение в степень массива.

`A.^B`

Поэлементное возведение массива в степень.
Если `A` и `B` являются скалярами, то `A` возводится в степень `B`.
Если `A` является скаляром и `B` является матрицей, матрица результатов

содержит элементы матрицы `A`, возведенные в степень, эквивалентную вхождениям в `B`.
Если `B` является скаляром и `A` является матрицей, то матрица результатов содержит элементы матрицы `A`, возведенные поэлементно в степень `B`.
Если `A` и `B` являются матрицами, то они обе должны иметь одинаковую размерность и матрица результатов содержит элементы матрицы `A`, возведенные в эквивалентные элементы матрицы `B`.

Смотреть: `help power`

`A'`

Транспонирование матриц.
Для вещественных матриц строки становятся столбцами, а столбцы – строками. Для комплексных матриц после этого все элементы заменяются комплексно-сопряженными числами.

Смотреть: `help ctranspose`

`A.'`

Транспонирование матриц.
Комплексные матрицы транспонируются без изменения их элементов.

Смотреть: `help transpose`

Для получения полного списка операторов MATLAB следует набрать `help ops` в окне командной строки. Можно получить описание поведения отдельного оператора, набрав `help operator_mnemonic`.

2.11. Перемещение

Система MATLAB обеспечивает перемещение по командам, введенных ранее пользователем, и редактировать их для дальнейшего использования. Список команд хранится в окне, расположенном в левой нижней части рабочей области системы. Можно дважды щелкнуть кнопкой мыши на любой из команды для повторного ее использования.

В дополнение, можно использовать клавиши вверх-вниз для перемещения по ранее введенным командам.

Существует сводная таблица команд для редактирования в командной строке, доступная через справку MATLAB. Вы также можете найти ее в Интернете, нажав на следующую ссылку и перейдя по ссылкам, в последовательности, указанной ниже: **http://www.mathworks.com/access/helpdesk/help/techdoc/matlab.html**:

- ✓ Getting Started
- ✓ Matrices and Arrays
- ✓ Controlling Command Window Input and Output

2.12. Использование точки с запятой (;)

В системе MATLAB точка с запятой "`;`" имеет два назначения. Она:

- ✓ определяет конец записи, например `A=[1,2; 3,4]`.
- ✓ указывает MATLAB на подавление вывода любых результатов выполнения команды на экран, когда строка заканчивается точкой с запятой (`;`). Это очень полезно когда генерируются большие матрицы или This is particularly useful when you generate large matrices or выполняются долгие скриптовые вычисления.

3. Упражнения

Часть 1

Написать скрипт в М-файле с именем `matrices.m`, который порождает результат,

приведенный ниже. Матричные результаты должны появляться один за другим, а не рядом как показано. Упражнение является последовательным, так что следует создавать каждую матрицу по порядку, как показано. Используйте заголовочные и встроенные комментарии так, как они используется в любом другом языке программирования.

1. Создайте следующие три матрицы в системе MATLAB:

```
A =              B =                          C =
   22   5          17   9   2   1   3            1   0   0
    4  11           6   4  11   9  16            0   1   0
   13   2                                        0   0   1
```

2. Используйте матрицу `A` из предыдущего пункта для создания следующих матриц (*используйте операции сложения, вычитания, умножения и транспонирования матриц*):

```
D =              E =                          F =
   22   5          66  15                       22   1  13
    1  11           3  33                        5  11   2
   13   2          39   6
```

3. Используйте матрицу `B` из предыдущего пункта для создания следующих матриц (*используйте операции сложения, вычитания и оператор двоеточие для матриц*):

```
G =                                          H =
   17   9   3   1   3                           3   3  17
    6   4   3   9  16                         16   3   6
```

4. Используйте матрицы `C` и `D` из предыдущих пунктов для создания следующих матриц (*используйте оператор двоеточия и операцию умножения для матриц*):

```
I =              J =                          K =
    1   0            1   0                      22   -5
    0   1            0  -1                       1  -11
                                                13   -2
```

Часть 2

1. Имеется изображение, которое сохранено в файле `image.dat` (это не правильный подход для хранения данных об изображении). Загрузите этот файл в систему MATLAB и затем отобразите изображение на экране, используя: `imshow(image)`
2. Это изображение является достаточно серым. Там мало разницы между светлым и темным. Мы исправим это с помощью матричных операций MATLAB к легко применимой Автоматической настройки контрастности, описанной в следующем уравнении:

```
image(i,j) = (image(i,j) - i_low) * 255 / (i_high - i_low)
```

   a. Во-первых, найдите информацию об интенсивности изображения:
      - сохраните минимальное значение интенсивности в переменной `i_low`;
      - сохраните максимальное значение интенсивности в переменной `i_high`;
      - просто для забавы, вычислите среднее значение и сохраните его в переменной `i_mean`.

b. Затем переместите интенсивность изображения к нижней части целевой шкалы интенсивности. Обычно это делает изображение темнее. Мы сделаем следующее:
   - вычтите `i_low` из изображения и сохраните результат в `image2`.

c. Далее, вычислите `k = 255 / (i_high - i_low)`

d. Затем, умножьте `image2` на `k` (вторая часть формулы выше) и сохраните результат в `image3`. Это расширит динамический диапазон. Вы должны заметить, что разница между наибольшим и наименьшим значением в настоящее время больше, чем это было раньше.

3. Если интересны ваши результаты, вы можете использовать:

```
figure, imshow(uint8(image2))
figure, imshow(uint8(image3))
```

Слово `'figure, '` позволяет вам отображать новое изображение в новом окне, так что можно увидеть все три изображения `image`, `image2` и `image3` одновременно.

Конечный результат

✓ Скрипт с именем `matrices.m` с вашим решением задачи с матрицами, описанной в части 1.

✓ Скрипт с именем `autocontrast.m` с вашим решением задачи с контрастностью, описанной в части 2.

Помните, что необходимо обеспечить пояснительную информацию и корректные комментарии к вашему коду.